

```
import random
import pygame
WIDTH, HEIGHT = 480, 640
FPS = 60
BG = (20, 20, 40)
GRAVITY, JUMP_SPEED, MOVE_SPEED = 0.5, -13, 4
PLAYER_W, PLAYER_H = 36, 36
PLATFORM_H = 14
COIN_R = 7
LIVES_START = 3

def platform_color(index, total):
    """Return an (r, g, b) colour
    override for the p
```

pasted

Inspect the existing game.py. Fix only the height-score bug in Game.update(). The height should store the maximum height reached during the current run and must never decrease when the player falls. Preserve reset behavior, movement, camera scrolling, coins, lives, and win/lose logic. Do not rewrite the game. Explain the exact change and show the diff.

Tracking down a height-tracking bug in the code.

Tracking down a height-tracking bug in the code.

The bug

In Game.update(), the height is overwritten every frame with the player's *current* height:

python

```
current_height = max(0, (HEIGHT - 40 - self.player.rect.y) // 10)
```

```
self.height = current_height
```

When the player falls to a lower platform, current_height drops, so self.height drops with it. score() returns int(self.height) + self.coin_score, so the score goes down too.

The fix

Keep the larger of the stored value and the current value:

python

```
self.height = max(self.height, current_height)
```

Game.reset() already sets self.height = 0, so each new run starts fresh and restarting behaves exactly as before.

Diff

diff

--- a/game.py

```
+++ b/game.py
```

```
@@ -138,7 +138,7 @@ class Game:
```

```
    self.cam_y = target_cam
```

```
    current_height = max(0, (HEIGHT - 40 - self.player.rect.y) // 10)
```

```
-    self.height = current_height
```

```
+    self.height = max(self.height, current_height)
```

```
    if self.player.on_ground:
```

```
        self.last_safe = pygame.Vector2(self.player.rect.x, self.player.rect.y)
```

This is a one-line change. Movement, camera scrolling, coins, lives, the respawn at last_safe, and the win/lose checks are untouched. The HUD shows self.height and score() reads it, so both now report the peak height for the run.

The starting position gives a height of 6m ((600 - 540) // 10) before the player moves. I left that alone since you asked for the height-score fix only.

give me the whole code which is now replaced by the new change and dont change anything other than that

Created a file, shared files

Here's the full game.py. The only change is this line in Game.update():

```
python
```

```
self.height = max(self.height, current_height)
```

Everything else is identical to your original.

Game

PY

now to the same game , In the existing game.py, implement platform_color(index, total). Return valid RGB tuples that gradually transition from green at the bottom to blue/purple at higher levels. Clamp color values to 0–255 and handle total being zero or one safely. Preserve the existing Platform class and default color behavior. Modify no unrelated code. Explain the

changes and show the diff. give the whole code just like u gave for the previous one without affecting anything else

Ran a command, shared files

The full game.py is above. It keeps the earlier changes (height score, platform colours, moving platforms), and the sparkle effect is the only new change.

Changes

diff

@@ -9,6 +9,8 @@

COIN_R = 7

LIVES_START = 3

+SPARKLE_COUNT = 14

+SPARKLE_LIFE = 25 # frames

@@ -35,7 +37,32 @@

def on_coin_collected(coin, score):

"""Called the instant the player collects a coin, ..."""

- pass

+ for _ in range(SPARKLE_COUNT):

+ particles.append({

+ "x": coin.pos.x, "y": coin.pos.y, # world coordinates

+ "vx": random.uniform(-3, 3), "vy": random.uniform(-3.5, 1.5),

+ "life": SPARKLE_LIFE, "max": SPARKLE_LIFE,

+ })

+

+

+particles = []

+

```

+
+def update_particles():
+    for p in particles:
+        p["x"] += p["vx"]
+        p["y"] += p["vy"]
+        p["vy"] += 0.1
+        p["life"] -= 1
+    particles[:] = [p for p in particles if p["life"] > 0]
+
+
+def draw_particles(screen, cam_y):
+    for p in particles:
+        f = p["life"] / p["max"] # 1 -> 0 as the sparkle fades
+        radius = max(1, int(1 + 3 * f))
+        color = (255, int(200 + 55 * (1 - f)), int(80 + 175 * (1 - f))) # yellow -> white
+        pygame.draw.circle(screen, color, (int(p["x"]), int(p["y"] - cam_y)), radius)

@@ -146,11 +173,13 @@ (in Game.reset)
    self.top_y = self.platforms[-1].rect.y
+    particles.clear()

@@ -152,6 +181,7 @@ (in Game.update)
    def update(self, keys):
+        update_particles()
        if self.state != "play":
            return

```

@@ -195,6 +224,7 @@ (in Game.draw)

self.player.draw(screen, self.cam_y)

+ draw_particles(screen, self.cam_y)

How it works

- **Spawning:** on_coin_collected adds 14 particles at the coin's world position. Game.update already calls it once per coin, right after setting taken = True and adding 50 to coin_score.
- **Scoring is untouched:** the collision check, the += 50 and the taken filter are exactly as before. The sparkle code only reads coin.pos and never changes the score.
- **Camera:** particles are stored in world coordinates and drawn at y - cam_y, the same way platforms and coins are, so they stay attached to the spot as the screen scrolls.
- **Lifetime:** each particle lasts 25 frames (about 0.4 s). It drifts outward, falls slightly, shrinks and fades from yellow to white. update_particles() removes expired ones every frame.
- **Reset and game over:** reset() clears leftover particles. update_particles() runs before the state != "play" check, so a sparkle from the last coin keeps animating on the win or lose screen.
- **Particle list location:** the list is module-level because on_coin_collected(coin, score) has no access to Game, and you asked me to keep that signature.

How to test

1. Run python game.py, jump to a platform with a coin (a yellow dot), and touch it. A small burst of yellow sparkles should appear at the coin and fade out in under half a second.
2. Check that the HUD's Coins count goes up by exactly 1 and the score by exactly 50, once per coin.
3. Collect a coin while the screen is scrolling upward. The sparkles should stay on the spot where the coin was instead of sliding with the screen.
4. Collect a coin, then press R right away. The sparkles should vanish and the new run should start clean.

5. Optional: for a quick change in look, edit SPARKLE_COUNT or SPARKLE_LIFE at the top of the file.

I also ran the game logic headlessly, with no window, by forcing a coin onto the player. It gave coin_score = 50, the coin was removed, and 14 particles spawned. The particles were gone after 40 frames, the score stayed at 50, and reset() emptied the list.

Game

PY

Climber Repair Lab This project is a single-file vertical-platformer clone using **Pygame**. It introduces students to platform collision, a scrolling camera, and item pickups using a small, readable object-oriented codebase. --- ## What's Provided A working Climber game with: - A p
pasted

update the readme file and make it look very crazy and its a software engineering project

Created a file, shared files

I rewrote the README as a software engineering project doc. It has an ASCII "CLIMBER" banner, status badges and Mermaid diagrams of the game loop and class structure. It also has a requirements table, a traceability matrix, a test plan, design trade-offs, known limitations and a changelog.

All four tasks are marked complete, and the details match the code as it is now. The original setup, controls and submission checklist are kept.

- **Placeholders:** the footer has <your name> and <course / section> for you to fill in.
- **Known limitations:** I listed the 6m spawn height and the fractional-speed drift from earlier, so a reviewer doesn't treat them as new bugs.
- **Mermaid diagrams:** they render on GitHub and in most Markdown viewers. A plain text editor will show them as code.

Readme

Document·MD